

C#, .NET i Visual Studio

Bartosz Miąskowski

Czym jest .NET?

Definicja:

.NET to platforma programistyczna stworzona przez Microsoft, która umożliwia tworzenie i uruchamianie aplikacji w różnych językach programowania (np. C#, F#, VB.NET).



.NET

Czym jest .NET?

Główne cechy:

- Umożliwia tworzenie **aplikacji webowych, desktopowych, mobilnych, chmurowych i IoT**.
- Działa na wielu systemach operacyjnych: **Windows, Linux, macOS**.
- Zawiera zestaw bibliotek i narzędzi ułatwiających tworzenie nowoczesnego oprogramowania.

Czym jest .NET?

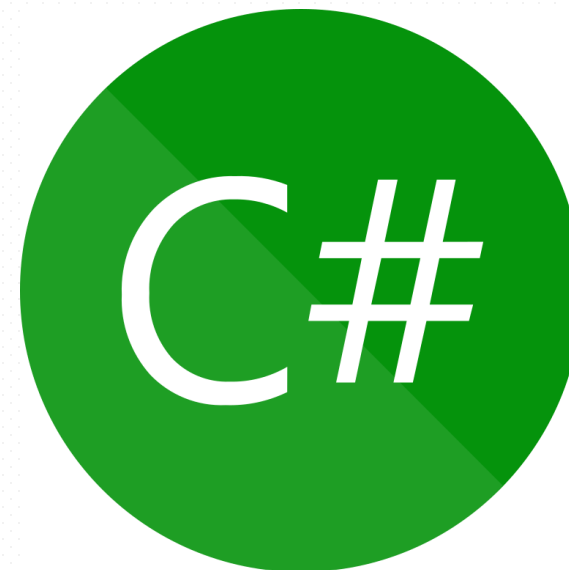
Główne cechy:

- Umożliwia tworzenie **aplikacji webowych, desktopowych, mobilnych, chmurowych i IoT**.
- Działa na wielu systemach operacyjnych: **Windows, Linux, macOS**.
- Zawiera zestaw bibliotek i narzędzi ułatwiających tworzenie nowoczesnego oprogramowania.

Od 2016 roku platforma .NET jest **otwartoźródłowa (open-source)** i rozwijana we współpracy ze społecznością na GitHubie.

Czym jest C#?

C# to nowoczesny, obiektowy język programowania stworzony przez firmę **Microsoft**. Został zaprojektowany jako język prosty w użyciu, bezpieczny i wydajny, ściśle powiązany z platformą **.NET**.



The modern, innovative, open-source programming language for building all your apps.

Dlaczego C#?

C# to nowoczesny, innowacyjny, otwartoźródłowy i wieloplatformowy język programowania obiektowego, należący do piątki najpopularniejszych języków na GitHubie.

- Język łatwy w nauce.
- Pozwala na tworzenie wielu rodzajów aplikacji.
- Pełne bezpieczeństwo typów.
- Charakteryzuje się wysoką wydajnością.

Paradygmat

C# został zaprojektowany tak, aby **nie ograniczać programisty do jednego stylu programowania**. Wspiera wiele paradygmatów, co czyni go niezwykle elastycznym i uniwersalnym.

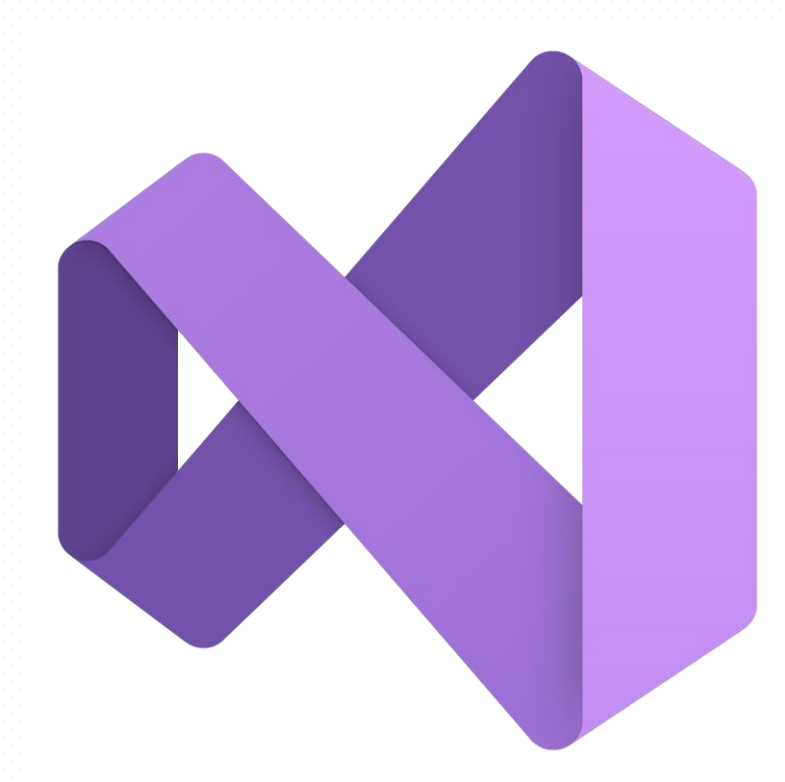
Paradygmaty wspierane przez C#:

- Programowanie obiektowe (OOP) – klasy, dziedziczenie, polimorfizm, enkapsulacja.
- Programowanie funkcyjne – funkcje wyższego rzędu, wyrażenia lambda, LINQ, niezmiennosc danych.
- Programowanie imperatywne – instrukcje wykonywane krok po kroku.
- Programowanie asynchroniczne – słowa kluczowe `async` / `await`.
- Programowanie deklaratywne – np. poprzez LINQ (opis co ma być zrobione, zamiast jak).
- Programowanie generyczne – typy i metody ogólne zwiększające reużywalność kodu.

Czym jest Visual Studio?

Definicja:

Visual Studio to zintegrowane środowisko programistyczne (**IDE**) stworzone przez **Microsoft**, służące do tworzenia, testowania i debugowania aplikacji w językach takich jak **C#**, **F#**, **C++**, **Python**, **JavaScript** i wielu innych.



Visual Studio

Możliwości:

- Edytor kodu z podpowiedziami (**IntelliSense**) i refaktoryzacją.
- Zintegrowany **debuger** do wykrywania błędów w czasie rzeczywistym.
- System budowania i testowania (np. MSTest, NUnit, xUnit).
- Projektowanie interfejsów graficznych (WinForms, WPF, MAUI).
- Integracja z Git, Azure, Dockerem i wieloma innymi narzędziami.

Visual Studio - wersje

- Visual Studio Community – bezpłatne, jeżeli firma nie przekracza 1 mln USD dochodu rocznie.
- Visual Studio Professional – 45 USD / użytkownika msc
- Visual Studio Enterprise – 250 USD / użytkownika msc

Visual Studio – porównanie wersji

Zintegrowane środowisko projektowe				^
Weryfikacja aktywnych zależności	—	—	✓	
Architektoniczne diagramy warstwowe	—	—	✓	
Weryfikacja architektury	—	—	✓	
Klonowanie kodu	—	—	✓	
CodeLens	✓	✓	✓	
Wgląd w definicję	✓	✓	✓	
Refaktoryzacja	✓	✓	✓	
Wdrażanie jednym kliknięciem w sieci Web	✓	✓	✓	
Model Resource Viewer	✓	✓	✓	
Wizualizowanie rozwiązań za pomocą wykresów zależności i map kodu		Może otwierać diagramy wygenerowane w innych wersjach programu Visual Studio w trybie tylko do odczytu.	Może otwierać diagramy wygenerowane w innych wersjach programu Visual Studio w trybie tylko do odczytu.	✓
Określanie wielu celów	✓	✓	✓	

Visual Studio – porównanie wersji

Zaawansowane debugowanie i diagnostyka

IntelliTrace	—	—	✓
Integracja mapy kodu z debuggerem	—	—	✓
Analiza zrzutu pamięci na platformie .NET	—	—	✓
Metryki kodu	✓	✓	✓
Debugowanie grafiki	✓	✓	✓
Statyczna analiza kodu	✓	✓	✓
Centrum wydajności i diagnostyki	✓	✓	Obejmuje funkcję profilowanie interakcji warstwy.
Snapshot Debugger	—	—	✓
Debugowanie z przechodzeniem w czasie (wersja zapoznawcza)	—	—	✓

Visual Studio – porównanie wersji

Narzędzia testowania

Live Unit Testing	—	—	✓
IntelliTest	—	—	✓
Microsoft Fakes (izolacja testu jednostki)	—	—	✓
Pokrycie kodu	—	—	✓
Unit Testing	✓	✓	✓

Tworzenie aplikacji dla różnych platform

Osadzone zestawy	—	—	✓
Zdalny symulator systemu iOS dla systemu Windows	✓	✓	✓
Natywni projektanci interfejsu użytkownika dla systemów iOS i Android	✓	✓	✓
.NET MAUI	✓	✓	✓

Historia .NET

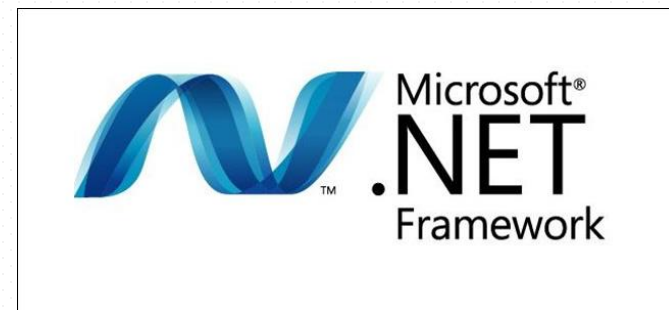
Początki:

- 1990s – Microsoft zaczyna prace nad nową platformą mającą ujednolicić tworzenie aplikacji dla Windows.
- 2002 – premiera .NET Framework 1.0 wraz z językiem C# i Visual Studio .NET. Umożliwia tworzenie aplikacji desktopowych i webowych w technologii ASP.NET.

Historia .NET

Ewolucja:

- 2005–2015 – kolejne wersje .NET Framework (2.0–4.8):
 - LINQ,
 - WPF,
 - WCF,
 - Entity Framework,
 - async/await.



Platforma staje się standardem dla aplikacji Windows, ale **ograniczona tylko do Windows.**

Historia .NET

.NET Core

2016 – Microsoft udostępnia .NET Core – open-source i wieloplatformową wersję .NET. Działa na Windows, Linux i macOS, rozwijana publicznie na GitHubie.



Historia .NET

.NET (po prostu)

2020 r – wydanie .NET 5, które łączy .NET Framework, .NET Core i Xamarin w jedną platformę.

2021–2025 r – kolejne wersje: .NET 6, 7, 8, 9, 10 rozwój MAUI, Blazora, konteneryzacji i AI.

Dzisiaj .NET to uniwersalna platforma do tworzenia nowoczesnych aplikacji.



.NET

Historia C#

Początki:

- Twórcą jest *Anders Hejlsberg* (ten sam, który wcześniej zaprojektował Turbo Pascal i Delphi).
- Powstał w roku 2000, wraz z premierą .NET 1.0.
- Inspirowano się językami: C, C++ i Java – C# miał odziedziczyć najlepsze cechy po tych językach: prostotę i wydajność.
- Nazwa języka jest inspirowana C++, C# to C++++ (cztery plusy, składają się w „płotek”)

C# - Etapy rozwoju

Wersja	Rok	Kluczowe zmiany
C# 1.0	2002	klasy, dziedziczenie, interfejsy, typy wartościowe, zarządzanie pamięcią przez GC
C# 2.0	2005	typy generyczne, anonimowe metody, iteratory (yield)
C# 3.0	2007	LINQ, wyrażenia lambda, typy anonimowe, var
C# 4.0	2010	dynamic, opcjonalne i nazwane parametry
C# 5.0	2012	async / await – rewolucja w programowaniu asynchronicznym
C# 6.0	2015	interpolacja stringów, wyrażenia ciała (expression-bodied members)
C# 7.x	2017	tuples, pattern matching, local functions, ref returns
C# 8.0	2019	nullable reference types, switch expressions, async streams
C# 9.0	2020	records, init-only properties, pattern matching v2
C# 10-13	2021-2025	global usings, file-scoped namespaces, list patterns, primary constructors, udoskonalenia dla AI i cloud-native apps

C# dzisiaj

- Jest jednym z najbardziej rozwiniętych języków ogólnego zastosowania,
- aktywnie rozwijany przez Microsoft i społeczność open-source,
- rdzeniem ekosystemu .NET – używanym w aplikacjach webowych, desktopowych, mobilnych, chmurowych, a nawet grach.

Historia Visual Studio

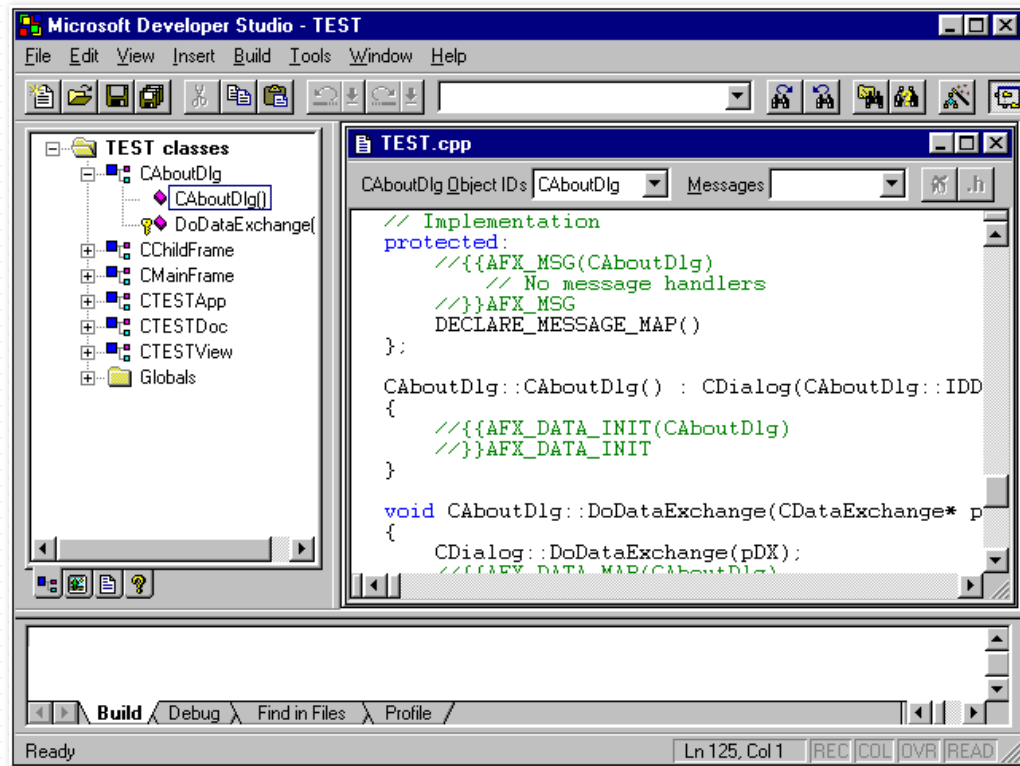
Początki:

- Rok 1997 – pierwsze wydanie Visual Studio 97Zintegrowało różne produkty Microsoftu (Visual Basic, Visual C++, Visual FoxPro) w jedno środowisko IDE.
- Celem było uproszczenie pracy programistów i ujednolicenie tworzenia aplikacji dla Windows.

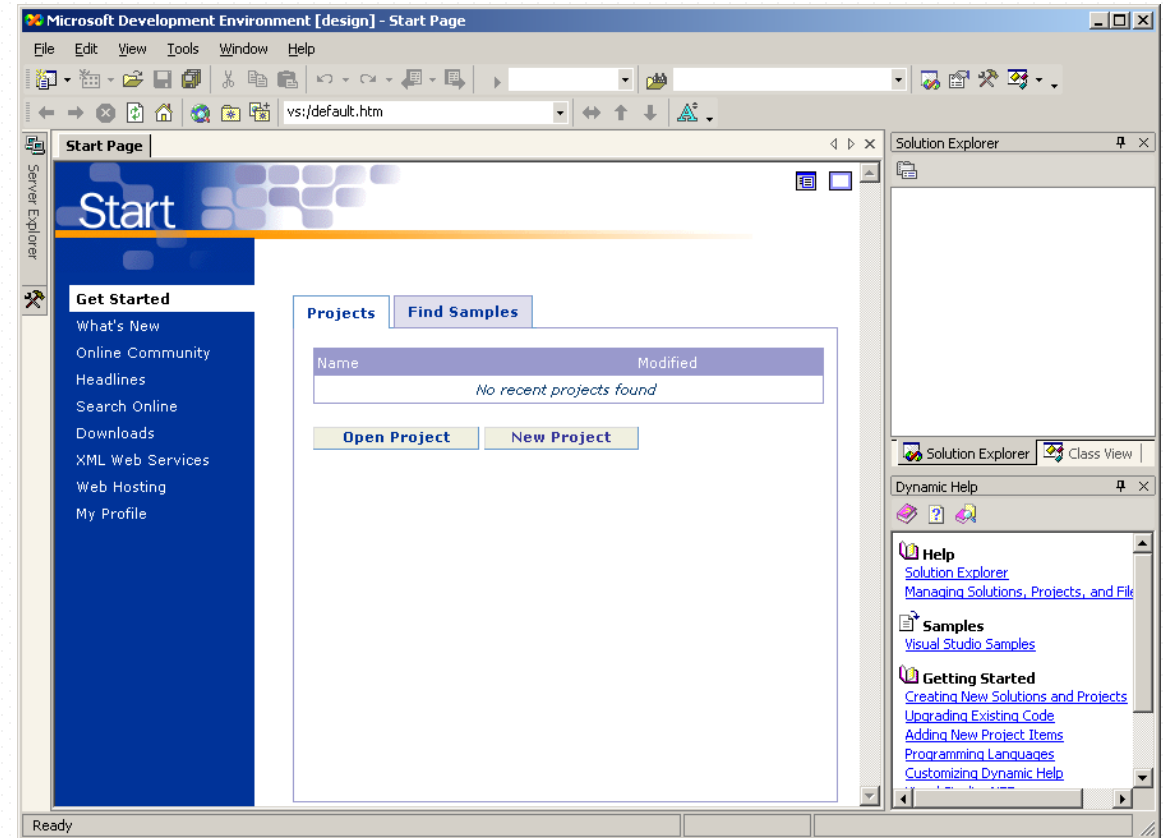
Rozwój Visual Studio

Wersja	Rok	Kluczowe wydarzenia
Visual Studio .NET (VS 2002)	2002	Wprowadzenie języka C# i platformy .NET Framework 1.0
VS 2005 / 2008	2005–2008	Nowe wersje .NET Framework, integracja z ASP.NET, Windows Forms, WPF
VS 2010	2010	Nowy interfejs (WPF), wsparcie dla .NET 4.0, TFS, rozszerzalność (Visual Studio Extensions)
VS 2013 / 2015	2013–2015	Integracja z Azure i Git, pierwsze kroki w stronę cross-platform
VS 2017	2017	Optymalizacja wydajności, obsługa .NET Core , Docker , Xamarin
VS 2019	2019	Ulepszony IntelliCode (AI), lepsza współpraca zespołowa, wydajność
VS 2022	2021	Pierwsze 64-bitowe Visual Studio , pełne wsparcie dla .NET 6 , nowy design
VS 2025	2025	Udoskonalony IntelliCode z AI, integracja z GitHub Copilot, wsparcie dla .NET 10 i projektów chmurowych

Tak wyglądało kiedyś...

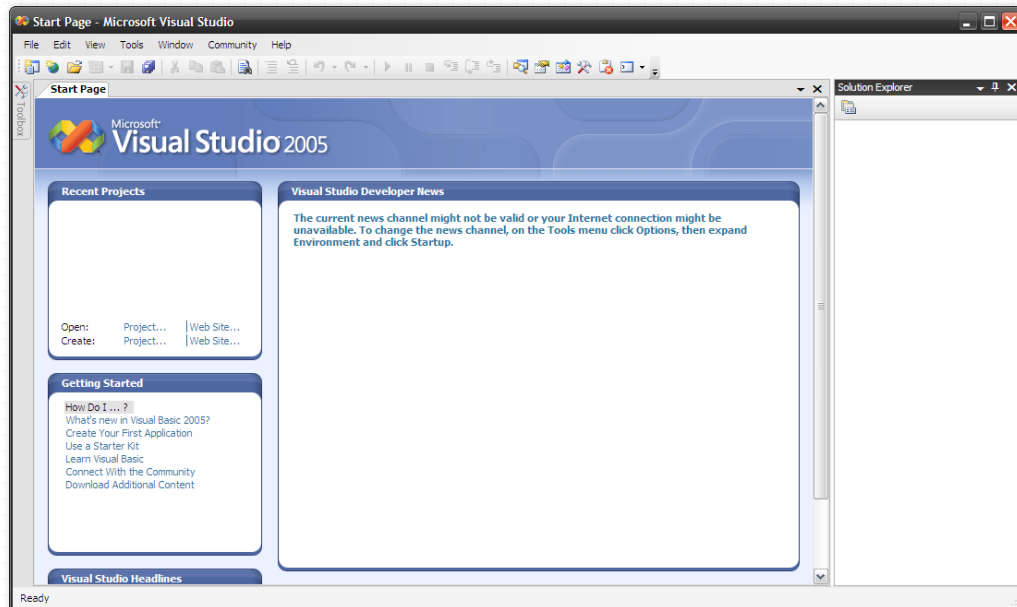


Visual Studio 97

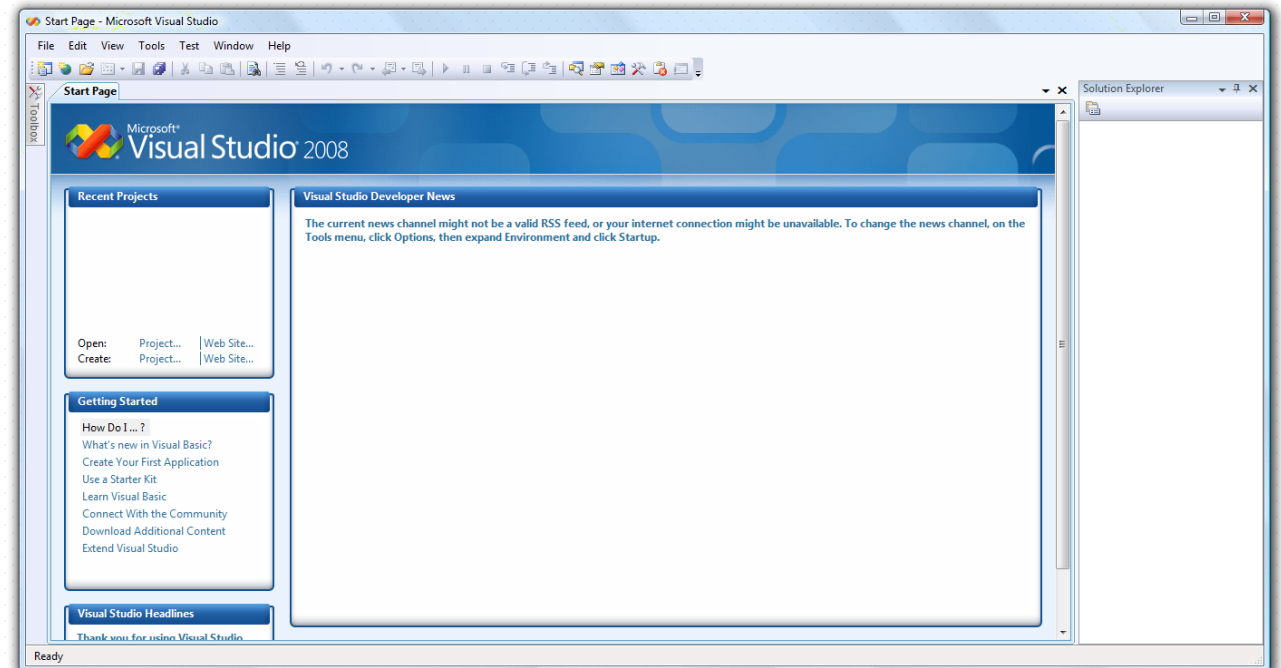


Visual Studio 2002

Tak to wyglądało kiedyś...

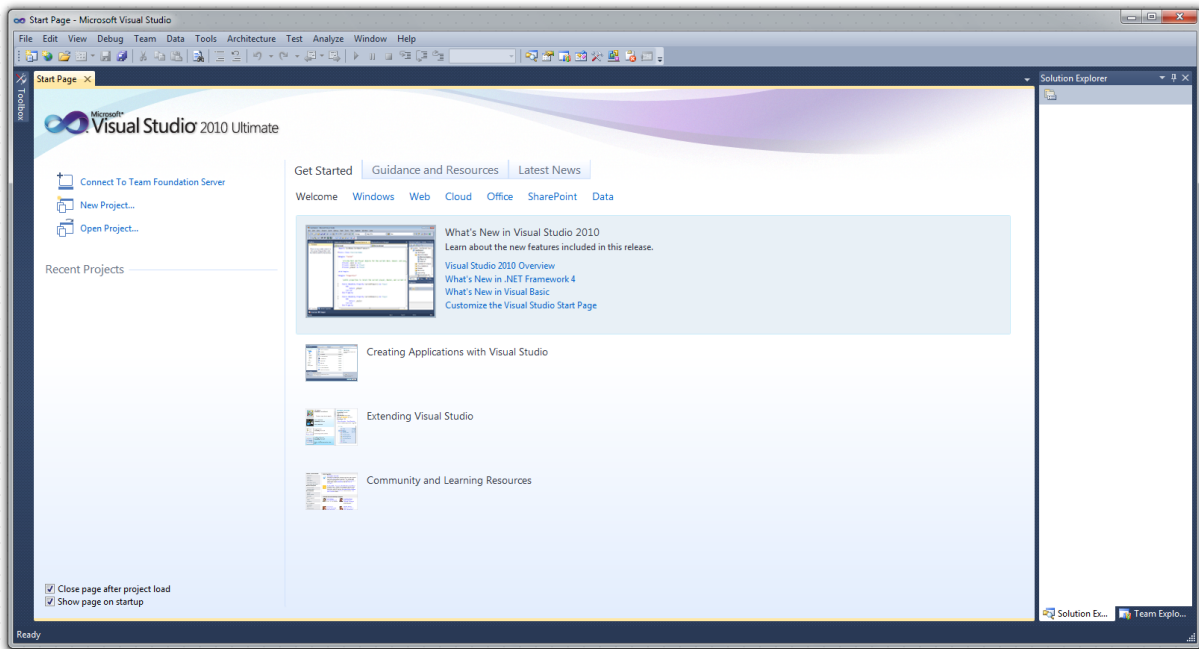


Visual Studio 2005

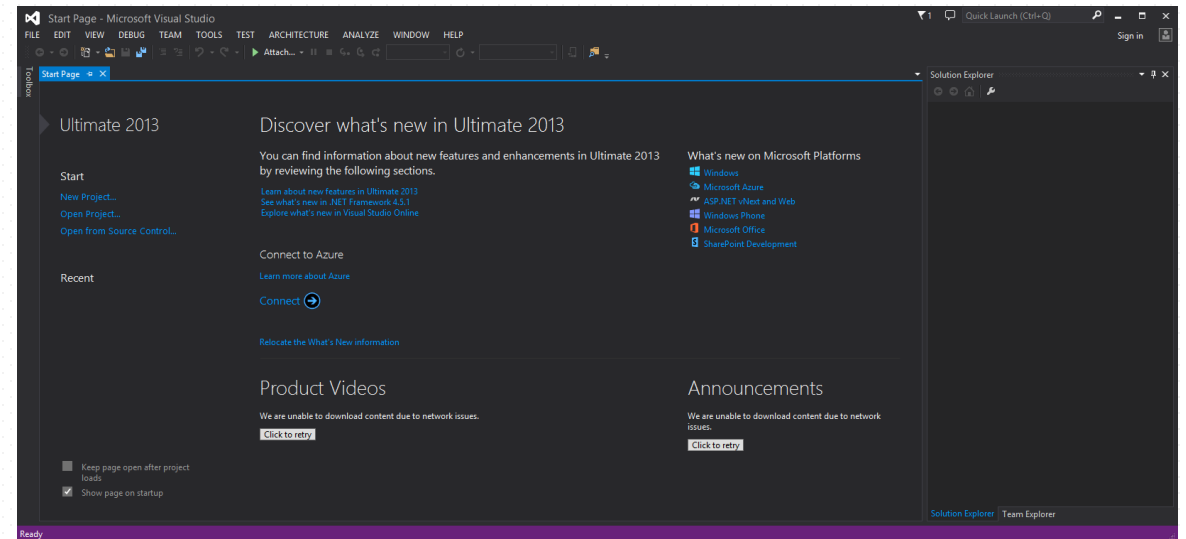


Visual Studio 2008

Tak to wyglądało kiedyś...

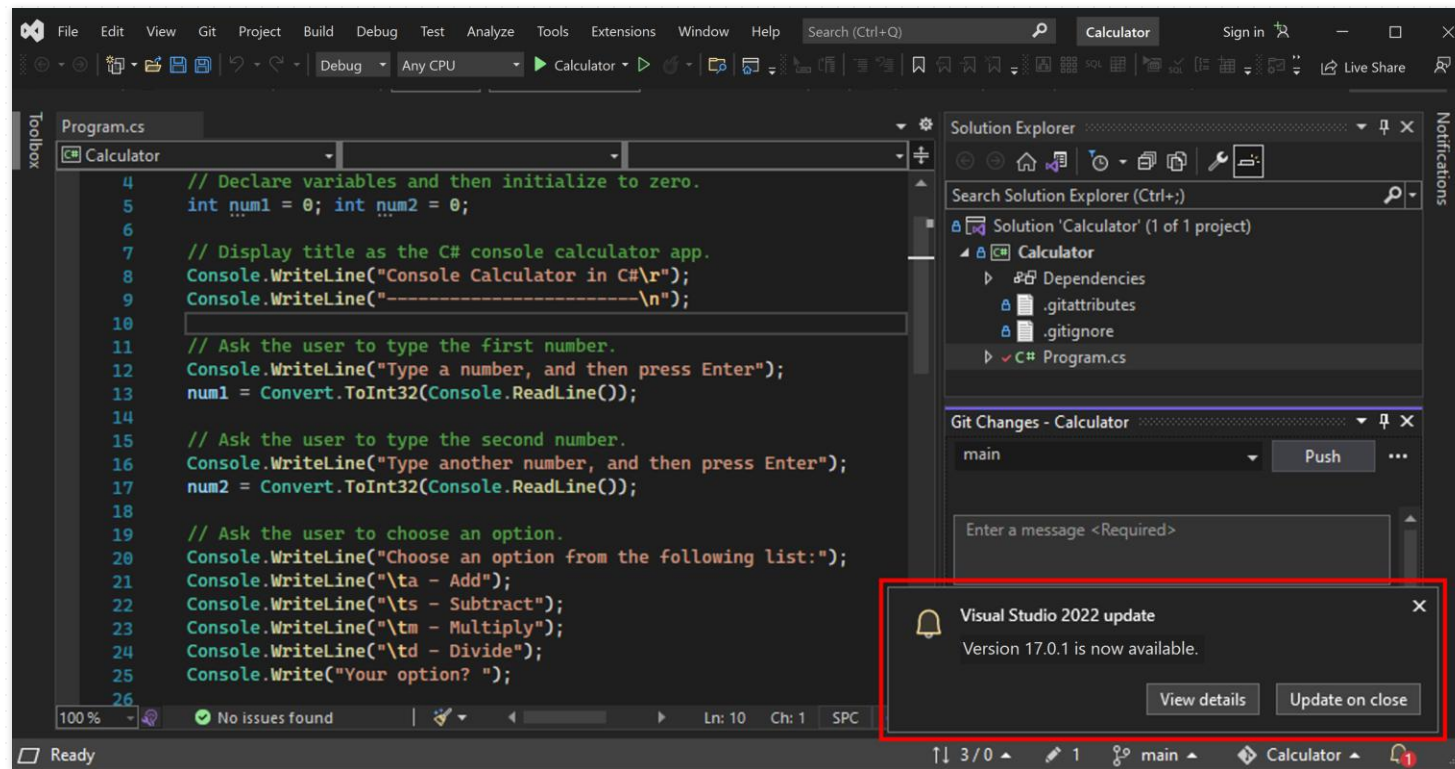


Visual Studio 2010



Visual Studio 2013

Tak wygląda dzisiaj



Kompilacja kodu C#

W C# kompilacja nie tworzy od razu kodu maszynowego (jak w C++). Zamiast tego powstaje kod pośredni (CIL), który później jest wykonywany przez środowisko CLR.

Kompilacja kodu C#

Etapy kompilacji:

1. Kod źródłowy (.cs)
2. Kompilacja do MSIL (Microsoft Intermediate Language)
 1. wykonywana przez kompilator Roslyn (csc.exe)
 2. wynik to plik .dll lub .exe zawierający kod pośredni i metadane
3. Uruchomienie w CLR (Common Language Runtime)
 1. MSIL jest tłumaczony just-in-time (JIT) do kodu maszynowego, odpowiedniego dla danego procesora
 2. CLR zarządza pamięcią, bezpieczeństwem, wyjątkami i wątkami

CLR

Common Language Runtime (CLR) – czyli **Wspólne Środowisko Uruchomieniowe** – to środowisko, które **uruchamia kod i dostarcza usługi**, ułatwiające proces tworzenia oprogramowania.

Brzmi jak kolejna niejasna definicja? Uściślijmy to przykładem:

Piszesz kod, kompilujesz go, a następny etap procesu jest już zadaniem CLR – środowisko to **kompiluje kod pośredni MSIL (z plików DLL lub EXE)** i tworzy środowisko, w którym Twój kod może zostać wykonany.

Możesz pisać kod w dowolnym języku programowania – tak długo, jak używany przez Ciebie kompilator (np. **C++/CLI, C#, Visual Basic, F#**) generuje kod zgodny z CLR.

Podczas pracy z platformą .NET często spotkasz się z pojęciem „**managed code**” – oznacza ono kod, którego wykonaniem **zarządza środowisko uruchomieniowe (CLR)**.

To ono odpowiada za **kompilację kodu zarządzanego do kodu natywnego** i jego wykonanie.

JIT

JIT (Just-In-Time) compilation – czyli **kompilacja „w locie”** – przekształca **kod pośredni MSIL** na **kod natywny na żądanie**, w momencie uruchamiania aplikacji, gdy zawartość zestawu (assembly) jest ładowana i wykonywana.

Ponieważ kompilacja odbywa się **„dokładnie w momencie potrzeby”**, ten element CLR często nazywany jest **JIT-erem** lub **kompilatorem JIT**.

Jak to działa:

JIT bierze pod uwagę fakt, że część kodu może **nigdy nie zostać wywołana** podczas działania programu. Zamiast więc zużywać czas i pamięć na przekształcenie całego kodu MSIL z pliku PE (Portable Executable) na kod natywny, JIT **kompiluje tylko te fragmenty**, które są rzeczywiście potrzebne w danym momencie.

Powstały w ten sposób kod natywny jest **przechowywany w pamięci**, aby przy kolejnych wywołaniach można było go **ponownie użyć bez rekompilacji** – co zwiększa wydajność działania aplikacji.

AOT

AOT (Ahead-of-Time compilation) to technika, w której **kod pośredni (MSIL) jest kompilowany do kodu natywnego jeszcze przed uruchomieniem aplikacji** – zamiast w trakcie, jak w przypadku JIT.

Jak działa?

- Kompilator AOT (np. **NativeAOT** w .NET 8) analizuje kod C#, usuwa nieużywane fragmenty (tzw. *trimming*) i tworzy **pojedynczy natywny plik wykonywalny (.exe)**.
- Dzięki temu aplikacja **uruchamia się szybciej** i ma **mniejsze zużycie pamięci**.

JIT vs AOT

Cecha

Moment kompilacji

Czas startu

Rozmiar aplikacji

Elastyczność

Zastosowania

JIT (Just-In-Time)

podczas uruchamiania programu

wolniejszy (kompilacja w locie)

większy (zawiera CLR i metadane)

pełna (dynamiczne ładowanie, refleksja)

aplikacje desktopowe, serwerowe

AOT (Ahead-of-Time)

przed uruchomieniem

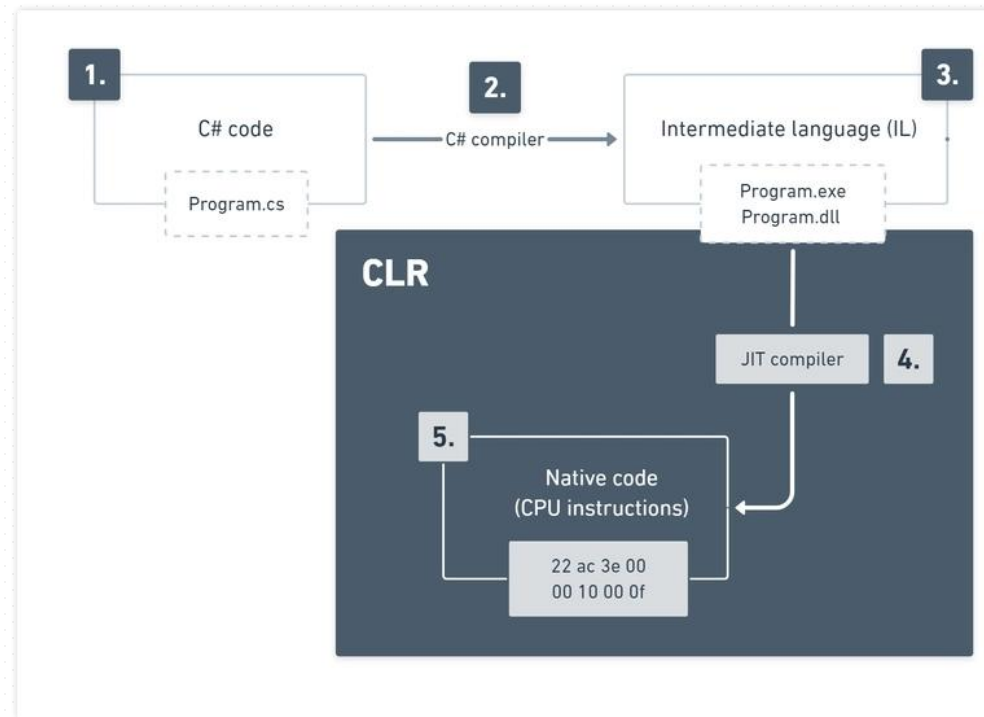
szybszy (gotowy kod natywny)

mniejszy po optymalizacji

ograniczona (często bez refleksji)

mikrouслуги, IoT, kontenery, narzędzia CLI

Kompilacja kodu C#



Kompilacja - porównanie

Język	Etapy kompilacji	Środowisko uruchomieniowe	Cechy charakterystyczne
C++	Kod źródłowy → bezpośrednio do kodu maszynowego	Brak (uruchamiany natywnie)	• Bardzo szybki • Brak zarządzania pamięcią • Zależny od platformy
Java	Kod źródłowy → bytecode (.class) → tłumaczony przez JVM (Just-In-Time)	JVM (Java Virtual Machine)	• Kompilacja do kodu pośredniego • Wysoka przenośność • JIT dla optymalizacji
C#	Kod źródłowy → CIL (Common Intermediate Language) → JIT / AOT przez CLR	CLR (Common Language Runtime)	• Dwustopniowa kompilacja • Zarządzanie pamięcią (GC) • Współpraca z różnymi językami .NET
Python	Kod źródłowy → bytecode (.pyc) → interpretowany przez Python Virtual Machine	Interpreter PVM	• Brak klasycznej kompilacji • Kod uruchamiany „w locie” • Mniejsza wydajność, ale duża elastyczność

Co możemy stworzyć z C# i .NET?

Typ aplikacji	Opis	Technologie
 Aplikacje desktopowe	Klasyczne aplikacje dla Windows, macOS, Linux	WinForms, WPF, .NET MAUI (desktop)
 Aplikacje webowe	Dynamiczne strony, API, mikroserwisy	ASP.NET Core, Blazor, Razor Pages, SignalR
 Aplikacje mobilne	Aplikacje na Androida i iOS z jednego kodu	.NET MAUI, dawniej Xamarin
 Aplikacje chmurowe	Usługi w chmurze, mikrouслуги, backend	Azure Functions, .NET + Docker/Kubernetes
 Aplikacje konsolowe / CLI	Narzędzia systemowe, automatyzacja	.NET Console App, NativeAOT
 Gry	Gry 2D/3D na PC, konsole, VR	Unity (C#), Godot (.NET)
 AI / ML / Data Science	Sztuczna inteligencja, uczenie maszynowe	ML.NET, TorchSharp, integracje z Pythonem i ONNX
 IoT / Embedded	Sterowanie urządzeniami, automatyka	.NET nanoFramework, .NET IoT
 Aplikacje serwerowe i integracyjne	API, systemy backendowe, integracje	ASP.NET Web API, gRPC, MassTransit